Software Architecture [H07Z9a]

Introduction and organization

Laurens Sion, Wouter Joosen

February 12, 2026

KU LEUVEN DistriNet

DistriNet, Dept. of Comp. Science, KU Leuven

Welcome

- Course instructors
  - Wouter Joosen
  - Laurens Sion
- Labs and project support
  - Laurens Sion (coordinator)
  - Stef Verreydt,
  - Willem Verheyen,
  - Anh-Duy Tran,
  - Qianying Liao
  - Jonah Bellemans

Overview of this lecture

1. Situating the course
2. Course objectives
3. Course organization
4. Key assumptions / expectations
5. First practical arrangements

(1) Situating the course

Software Engineering courses

- “Methodiek van de informatica” (MI)
  - Starter level programming course
- “Object-gericht programmeren” (OGP)
  - Advanced programming course – OO concepts
- “Ontwerp van Softwaresystemen” (SWOP/OSS)
  - First software engineering course (software design)
- “Vereistenanalyse voor complexe softwaresystemen”
  - Requirements analysis
- …. **Software Architecture**
  - Software engineering continued – *completed*?

Differences with "Software Design"
(or equivalent course)

- We further expand our view on the development life cycle
  - We work on requirements & analysis
  - We work on "early design": Software Architecture
- We evaluate architecture & design artifacts before implementation
  - No "*compiler comfort*"
  - AND: trade-offs
- We focus on non-functional requirements
  - Security, performance, availability, modifiability, fault tolerance, extensibility, adaptability, etc *… more on these later*

Differences with a “standard” software development course

Approximating reality:

- The requirements exist, to a large extent
- The target system
  - is an **extension** of an existing software system (existing technology),
  - has to **interoperate** with existing software systems
- The technological context cannot be determined from scratch
  Greenfield software development is rare
- We cannot ignore economics: cost, effort: software is typically more expensive than expected

(2) Course objectives

(2) Course objectives

- Broaden the scope: establish an understanding of the **entire software engineering process** in realistic software production settings
- To elaborate on the **role of analysis/evaluation** in this context, and to elaborate on the role of **software architecture** in the software engineering process
- To study the **creation** and **extension** of software architecture, and THUS to focus on **non-functional requirements**

Secondary objectives/additional skills

- **Team work**, team organization, collective decision-making, brainstorming, …
- **Time management**, planning, deadline-driven
- **Analytic**, top-down reasoning, addressing ambiguity, creating abstractions that work
- **Communicating** and reporting about technical decisions, motivating, discussing trade-offs
  - Incl. presenting, writing, processing and giving feedback
- **Modeling**, abstractions, using existing abstraction techniques (UML)
- **Peer review**, providing constructive feedback

(3) Course organization

Modus operandi

- **Project-driven work**

  experience course, intensive (!) yet manageable within the allocated time

  no emphasis on theory, but a grasp of key concepts is required

- **Teamwork**

  team size: **4** students

  time management, coordinated effort

- **Quality control**

  systematic delivery and feedback

Course activities

- 10 Lectures - Thursdays
  - Lecture plan will be published and announced on Toledo/Ultra (Course Documents > Lectures)
  - Includes 3 o4r Guest Lectures! (details to be discussed during Lectures 2 & 3 – February 19 & 26)
- 7 lab sessions - Mondays
  - Max! 6 sessions to work on project assignment (team),
  - And 1 session on using a tool (Visual Paradigm) and on modeling skills (individual) -- **09/03 – March 9**

  - Starting on **February 23!**

Course outline

- **Part 1** – Understanding the application case, defining **key requirements**
  - Architecturally-significant requirements (ASRs)
  - Quality attribute scenarios (QASs)
- **Part 2** – Extending a **software architecture**
  - Methodology & known principles/practices/tactics
  - Modeling
  - Making trade-off decisions, analyzing Architecture
- **Part 3** – Keep up with recent evolutions in **SDLC** (Software Development Life Cycle) and **software architecture**

Delineated project activities

- **Part 1**: requirements engineering – focus on ASRs and NFRs
- **Part 2**: extension from a starting point – scoped set of extensions
- **Part 3**: keeping up on important trends and evolutions in software engineering & software architecture (selected topics)

No blank pages to start from: we provide initial starting points

Timeline of the project

Domain description + Functional requirements + API specs

Requirement analysis

Deliverable 1: report

Non-functional requirements
Utility tree/ASRs
QASs

PART1

Initial architecture

"Take a tour of the initial architecture" lab session

"Taming the beast": working with Visual Paradigm lab session

Non-functional requirements

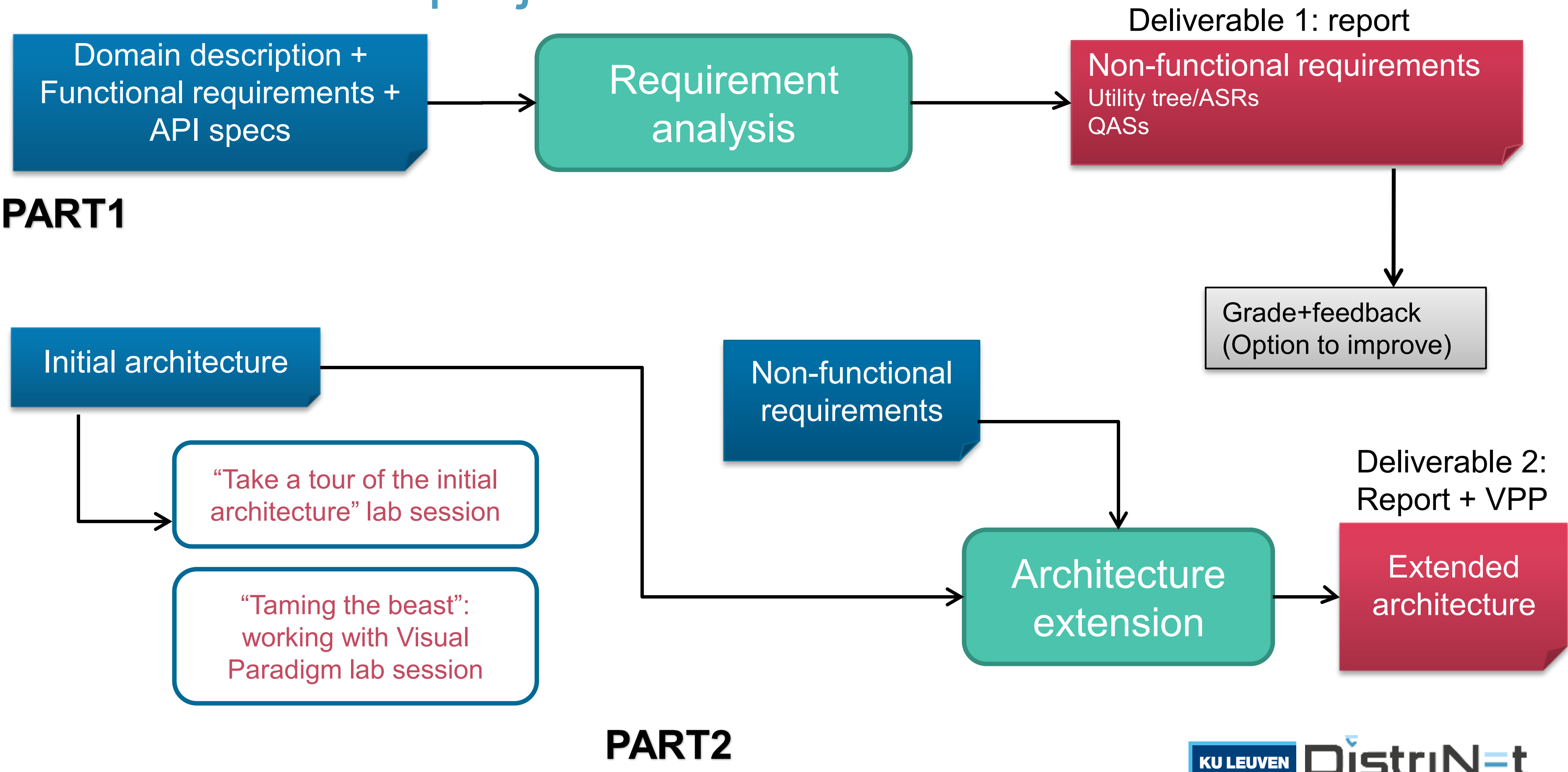
Architecture extension

Deliverable 2: Report + VPP

Extended architecture

PART2

Timeline of the project



<u>PART 3 is new from this year onwards: it enables reflection on important (recent and ongoing) trends in our engineering discipline, its innovations and its industry.</u>

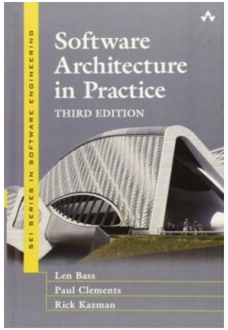
Continuous feedback

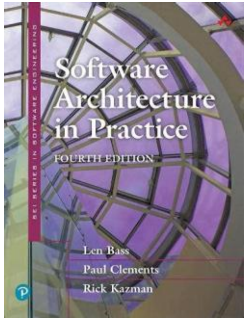
- Part 1: Grade+ feedback + option to resubmit improved version
- Part 2: SA Plugin for Visual Paradigm – *some* compiler comfort
- Part 2: Opportunity to ask questions in all sessions (Lab sessions & plenary lectures)

Continuous support – on-campus & online

- **Mondays**: Lab sessions = project working sessions
- **Thursdays**: Lectures
  - Theory in function of the assignment
  - Start with project Q&A
- **Other days**: Toledo discussion boards

Course Materials





- Slide hand-outs ! *Essential !*
- *This obviously includes the presentations of the guest*
- Books
  - “Software Architecture in Practice 3rd Ed.”, Bass, Clements, Kazman
  - “Pattern Oriented Software Architecture (Volume 4)” by Buschmann et al.

- Project support on Toledo and during lab sessions

Evaluation

- *Project evaluation sessions*
- Discussion of the project results (team-wise)
- Weights: 30% - 50% - 20%
  - Students must be successful in all parts

Course deadlines

- **Part 1: Requirements**
  About 3 weeks
  - Report: 30% of the score
    - [Feb 23 to **Wed, March 11 (noon)]**
- **Part 2: Architecture Extension**
  About 5 weeks
  - Report + vpp file: 50% of the score,
    - [Mar 23 - DL: **Wed, April 29 (noon)]**
- **Part 3: Short Presentation including answers to specific questions**
  - **We elaborate on this part in the lectures 2 &3….**

Project evaluation sessions affect grades (*obviously*) – possibly individually

How to contact us

1. **Content** - Toledo discussion boards: E.g.:General | Part1 | VP-UML | Part2 ….
2. **Team business** - deadlines, practical, organization
   SoftwareArchitecture@cs.kuleuven.be
   - Put **all team members** in CC
3. **Individual/personal issues**: mail to Laurens Sion **AND** Wouter Joosen

(decreasing order of preference)

- **Contacts** page on Toledo

(4) Key assumptions

Assumption/expectation I

- We expect students to have executed a software development project (similar to the software design course)

  - It is difficult to create/extend a software architecture if you are not (somewhat) experienced in building software

Assumption/expectation II

› Master students plan their efforts, and manage the plan

The SA project cannot be executed in “one intensive working session”: it requires iteration, reflection, and discussion

›› Teams that effectively collaborate

The SA project cannot be easily “split up in 4 parts, only to merge contributions at the very end”, *the puzzle pieces must fit well*

Coordinated team effort

Assumption/expectation III

- This course follows a project-centric approach
- We deliver what students need in order to be successful, but:
  - Pro-active attitude required
  - (Most of the) lectures are PART OF the project: lectures provide background knowledge that will help you understand

It is all about building up experience…

(so do not sit back, relax etc. …)

(5) First practical arrangements

Practicalities
1. Toledo

- All students: register to [H07Z9a] Software Architecture
  - (should be okay, but pls doublecheck)

Practicalities:
2. Toledo team registration

- To do: form and register a team (team size: **4**)
- The link: https://icts.kuleuven.be/apps/onebutton/registrations/1377834
  - **Lab session, pick a slot** either “10h30”, “13h30” or “16h”
  - Important for load balancing (online support / room capacity)

- **Find team members**: Discussion board is open

- Deadline: **Tuesday 17/02**

About team work

- Equal roles
  - Everyone should be equally familiar with the end result
  - Same amount of effort spent: keep track of the contributions of each team member
  - Time/effort reporting
  - Your contributions affect others (!)
  - Compare agendas

Problems and **team issues** > address | escalate **in time**

Software Architecture [H07Z9B]

Introduction and organization:
Final Time Table on February 19!

Laurens Sion

Wouter Joosen

DistriNet, Dept. of Comp. Science, KU Leuven